Gemini Chat

Cómo conectamos una web con la IA de Google AI Studio

Danilo Chambi, Ph.D. · 31 de octubre de 2026

Fundamentos Sociotécnicos de la Inteligencia Artificial, Aprendizaje Automático y Diseño Responsable

HTML CSS JavaScript API Gemini CloudPanel

EL PROYECTO

Una web que conversa con IA, sin servidor propio

Tres archivos y una API: eso es todo lo que hay detrás del chat.

Conversación real

El modelo recibe el historial y responde en streaming, palabra por palabra.

100 % frontend

HTML, CSS y JavaScript. Sin Node, sin base de datos, sin build.

Publicado

Desplegado como sitio estático en CloudPanel con HTTPS.

DESARROLLO

Así se construyó, paso a paso

1

Diseño

Interfaz de chat: mensajes, estados vacíos, configuración.

2

Estructura

HTML semántico y CSS responsive con tema oscuro.

3

Conexión

`fetch` a la API de Gemini y lectura del stream SSE.

4

Pruebas

Claves inválidas, modelos caídos, cuotas y respuestas vacías.

5

Despliegue

Sitio estático en CloudPanel + certificado SSL.

Cada paso se probó en el navegador antes de pasar al siguiente.

TECNOLOGÍAS

Con qué está hecho

HTML5

Estructura del chat, diálogo de configuración y accesibilidad.

CSS3

Tema oscuro, diseño responsive, animaciones y estados.

JavaScript

Estado, manipulación del DOM, `fetch` y streams.

API Gemini

Modelo de lenguaje de Google AI Studio vía HTTPS.

CloudPanel

Hosting estático en nginx con certificado Let's Encrypt.

Sin frameworks

Ni React ni bundler: solo tres archivos que se entienden de principio a fin.

ARQUITECTURA

Quién habla con quién



- La petición sale del navegador por **HTTPS** con la clave del visitante.
- La respuesta vuelve como **eventos SSE** y se pinta al instante.
- El servidor (CloudPanel) solo entrega los archivos: **no procesa la conversación**.

AUTENTICACIÓN

La API key

- Se obtiene gratis en **aistudio.google.com/apikey**.
- Se guarda en **localStorage**, solo en el navegador del visitante.
- **Nunca** se escribe en el HTML ni en el JavaScript.
- Cada persona que entra usa su propia clave.

> **Si la clave se publica en el código**, cualquiera puede leerla con «Ver código fuente» y gastar tu cuota de IA.

LA CONEXIÓN

La petición HTTP

```
POST https://generativelanguage.googleapis.com/v1beta/models/
     gemini-flash-latest:streamGenerateContent?alt=sse&key=•••
Content-Type: application/json

{
  "contents": [
    { "role": "user",  "parts": [{ "text": "Hola" }] },
    { "role": "model", "parts": [{ "text": "¡Hola!" }] }
  ],
  "generationConfig": { "temperature": 0.7 }
}
```

El historial se envía completo en cada petición: ahí está la «memoria» del chat.

STREAMING

La respuesta llega por trozos

- Google responde con **Server-Sent Events**.
- Cada línea `data: {...}` trae un fragmento de texto.
- Se lee con `ReadableStream` y `TextDecoder`.
- La interfaz se actualiza sin esperar el final.

```
// cada evento SSE
data: {"candidates":[{"content":
  {"parts":[{"text":"Hola"}]}}]}

// leer el stream
const reader = res.body.getReader();
const decoder = new TextDecoder();

decoder.decode(value, { stream: true });
onChunk(texto); // pintar
```

Resultado: el texto aparece mientras se genera, no después.

CONTEXTO

La memoria del chat la ponemos nosotros

1. Escribes
Se guarda como `user`

→

2. Se envía
Últimos 40 mensajes

→

3. Responde
Se guarda como `model`

- Los roles **alternan**: `user` → `model` → `user`.
- Más historial = mejor contexto, pero **más tokens** por petición.
- Por eso se recorta la ventana y se empieza siempre por un mensaje de usuario.

ROBUSTEZ

Qué pasa cuando algo falla

`400` clave o petición inválida | `403` acceso denegado | `404` modelo retirado | `429` cuota agotada | `503` modelo saturado

- Los errores se traducen a **mensajes claros**, no a JSON crudo.
- Botón **Reintentar** y botón **Detener** (`AbortController`).
- Si el modelo da 503 o 404, la app **prueba otro automáticamente**.

SEGURIDAD Y LÍMITES

Lo que hay que tener claro

Bien resuelto

- Clave fuera del código fuente.
- Cada visitante usa la suya.
- HTTPS en el despliegue.
- Sin backend que atacar.

Con cuidado

- La clave es visible en DevTools.
- No vale para producción multiusuario.
- La IA puede inventar datos.
- La calidad depende del modelo y la temperatura.

Para producción: mover la llamada a un backend que guarde la clave en el servidor y actúe de proxy.

DESPLIEGUE

Publicado en CloudPanel

1

Crear el sitio

«Create a Static HTML Site» con el dominio.

2

Subir archivos

Los tres archivos a `htdocs/tudominio.com`.

3

Activar SSL

Let's Encrypt desde el panel.

4

Probar

Abrir el dominio, pegar la clave y chatear.

Sin build, sin base de datos, sin reiniciar servicios: nginx sirve los archivos al instante.

DEMO

Preguntas para probarlo en clase

- «Explícame la recursión con una analogía cotidiana.»
- «Escribe una función en JavaScript que valide emails y explícala.»
- «Resume este texto en tres puntos: …» (pegar un párrafo)
- «Dame 3 argumentos a favor y 3 en contra de prohibir el móvil en clase.»
- «Ahora hazme 5 preguntas tipo test sobre lo anterior.» (muestra la memoria)

> **Truco:** cambia la temperatura a 1.5 y repite la misma pregunta para comparar respuestas.

CONCLUSIONES

Lo que aprendí

- **Una API** es una URL, unos datos y una autenticación.
- **El streaming** convierte una espera en una conversación.
- **El historial** es la memoria del chat.
- **Errores y seguridad** son parte del proyecto, no un extra.

Gracias · ¿Preguntas?